

PCS3732 - Telemetria e Gêmeo Digital para Detecção de Acidentes

Identificação do Grupo

Integrante 1: Gabriel Soares da Silva

NUSP 1: 11808234

Integrante 2: Pedro Guilherme Croitor da Cruz

NUSP 2: 13685002

Integrante 3: Thaylor Hugo Araujo da Silva

NUSP 3: 13684425

1. Motivação e Justificativa

Os acidentes de trânsito envolvendo motociclistas representam uma das principais causas de trauma grave no transporte urbano. Devido à ausência de uma estrutura de proteção ao redor do condutor, a rapidez no socorro médico após uma colisão ou queda é o fator determinante para a sobrevivência e redução de sequelas. No entanto, em rodovias ou locais isolados, o tempo de resposta pode ser severamente prejudicado pela incapacidade da vítima de solicitar ajuda.

Projetos e sistemas consolidados na indústria automotiva e de tecnologia vestível validam a eficiência do monitoramento cinemático contínuo para essa finalidade:

- **eCall (União Europeia):** Sistema obrigatório em veículos na Europa que realiza chamadas automáticas de emergência em caso de colisões graves, transmitindo dados de localização e horário (ver [eCall Initiative](#)).
- **Detecção de Acidentes (Crash Detection - Apple/Google):** Algoritmos em smartwatches e smartphones que utilizam giroscópios de alto alcance dinâmico (high-g) e acelerômetros para identificar picos de desaceleração e variações de atitude característicos de impactos veiculares (ver [Apple Crash Detection Technology](#)).

O Protótipo

Este projeto propõe o desenvolvimento de um protótipo IoT compacto de telemetria e Gêmeo Digital (Digital Twin) em tempo real. O dispositivo é fixado ao veículo/condutor para capturar dados continuamente. A lógica de detecção baseia-se no padrão cinemático característico de acidentes de motocicleta:

- Pico de aceleração ou desaceleração abrupta (impacto frontal/lateral ou frenagem de emergência);

- Queda imediata da velocidade linear (parada repentina do veículo);
- Movimentos angulares caóticos ou inclinação excessiva persistente (Roll/Pitch) que indicam tombo ou rotação anômala do condutor fora da posição vertical normal.

Com o processamento desses dados na borda via Raspberry Pi e a transmissão via WebSocket para um Dashboard com modelo 3D em tempo real, o sistema demonstra a viabilidade de monitoramento cinemático contínuo para suporte à decisão e alertas de emergência.

2. Objetivos

2.1 Objetivo Geral

Desenvolver um sistema embarcado de telemetria inercial e visualização em tempo real capaz de capturar, filtrar e transmitir a cinemática de um protótipo móvel via Wi-Fi/WebSocket, permitindo o monitoramento de orientação e a identificação visual de padrões cinemáticos associados a impactos e quedas.

2.2 Objetivos Específicos

- **Integração de Hardware:** Conectar e calibrar a unidade de medição MPU6050 ao barramento I2C da Raspberry Pi.
- **Filtragem:** Implementar algoritmo de Filtro Complementar para estimativa estável de atitude (Roll, Pitch e Yaw) e algoritmo de estimativa de velocidade com ZUPT (Zero Velocity Update) para atenuação de drift.
- **Comunicação sem Fio:** Configurar a Raspberry Pi como um Ponto de Acesso Wi-Fi autônomo e disponibilizar um servidor Socket.IO para broadcast contínuo de dados.
- **Gêmeo Digital e Dashboard:** Desenvolver uma aplicação Web (React + Three.js) que consuma a telemetria a 50 Hz e renderize a orientação e movimento do protótipo em 3D de forma fluida.

3. Especificação de Requisitos

3.1 Requisitos Funcionais (RF)

ID	Descrição
RF01	O sistema deve ler continuamente os dados de aceleração e velocidade angular do MPU6050.
RF02	O sistema deve calibrar automaticamente os offsets (biases) dos sensores na inicialização em repouso.
RF03	O sistema deve calcular a orientação espacial (Roll, Pitch e Yaw) em graus utilizando Filtro Complementar.

ID	Descrição
RF04	O sistema deve estimar a velocidade linear do protótipo removendo o vetor de gravidade e aplicando ZUPT quando em repouso.
RF05	A Raspberry Pi deve atuar como servidor WebSocket, transmitindo os pacotes de telemetria.
RF06	A aplicação Web (Dashboard) deve se conectar à rede Wi-Fi gerada pelo protótipo e consumir o evento de telemetria em tempo real.
RF07	A interface Web deve exibir um modelo 3D (Gêmeo Digital) cuja rotação espacial acompanhe os ângulos recebidos.
RF08	A interface Web deve permitir alternar entre o modo de simulação (Mock) e conexão real com o dispositivo físico.

3.2 Requisitos Não Funcionais (RNF)

ID	Categoria	Descrição
RNF01	Frequência de Amostragem	O loop de amostragem e filtragem na Raspberry Pi deve rodar a 50 Hz.
RNF02	Latência	A latência média de transmissão via no Hotspot local deve ser inferior a 100 ms para garantir fidelidade ao movimento real.
RNF03	Portabilidade / Mobilidade	O módulo embarcado (Raspberry Pi + MPU6050) deve operar alimentado por bateria (Power Bank) sem cabos de energia.
RNF04	Confiabilidade I2C	O código de leitura deve tratar erros do barramento I2C, descartando quadros ruidosos sem interromper o loop.
RNF05	Suavidade Visual (UI)	A renderização do modelo 3D no Three.js deve utilizar interpolação linear, evitando re-renders custosos a cada novo pacote.
RNF06	Autonomia de Rede	O protótipo deve criar sua própria rede Wi-Fi com IP fixo.

4. Arquitetura

4.1 Visão Geral e Diagrama de Blocos

O sistema é dividido em dois subsistemas comunicados via WebSocket: um backend embarcado (Raspberry Pi + IMU MPU6050), e um frontend Web (React + Three.js), que consome a telemetria e renderiza o Gêmeo Digital. A Figura 1 mostra o fluxo de dados entre esses componentes.

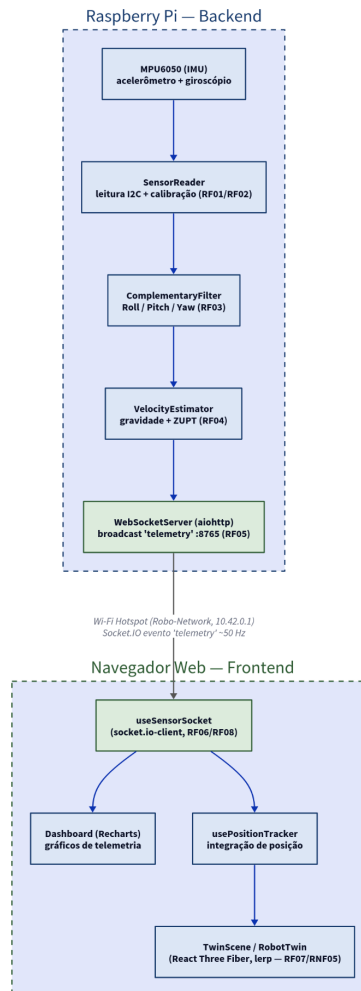


Figura 1 — Diagrama de blocos da arquitetura proposta.

4.2 Diagramas de Sequência

A Figura 2 mostra a troca de mensagens em um ciclo de telemetria: leitura, fusão no filtro, estimativa de velocidade, broadcast via Socket.IO e atualização do Gêmeo Digital. Esse ciclo se repete continuamente a 50 Hz.

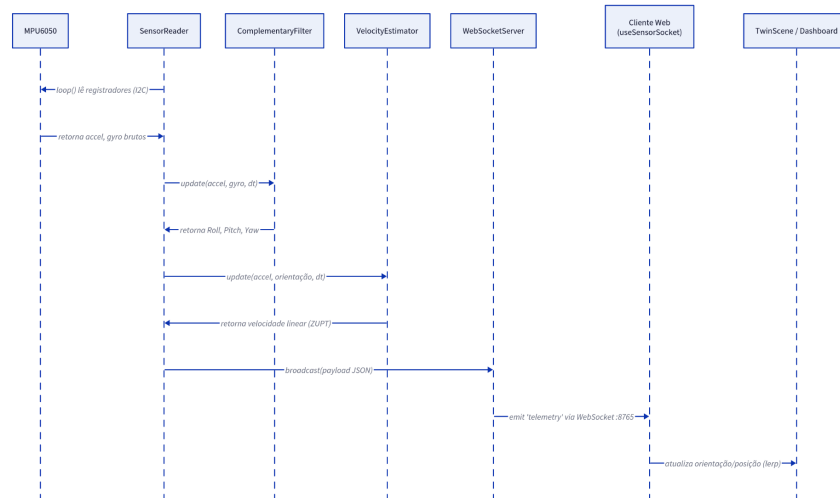


Figura 2 — Diagrama de sequência de um ciclo de telemetria.

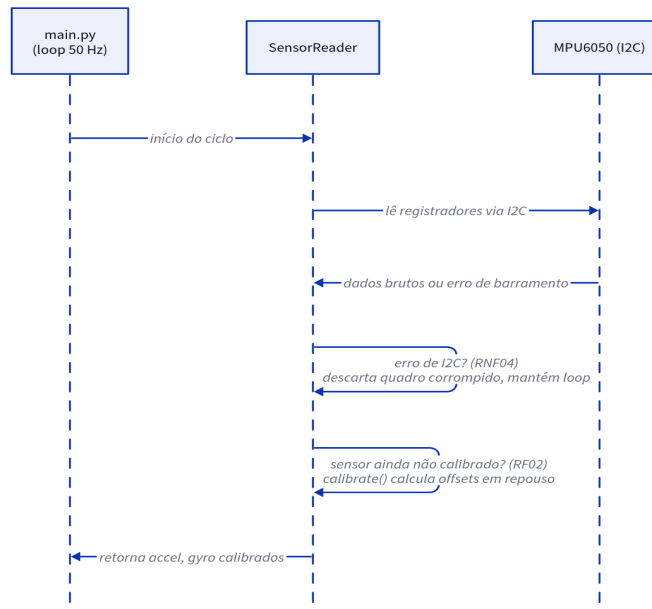


Figura 3 — Diagrama de sequência de um ciclo de calibração e tratamento de erro I2C.

4.3 Fluxograma do Loop Principal

A Figura 3 descreve a lógica do loop principal (main.py): tratamento de falhas I2C (RNF04), calibração inicial (RF02), Filtro Complementar (RF03), correção ZUPT (RF04) e controle do período de 20 ms (RNF01).

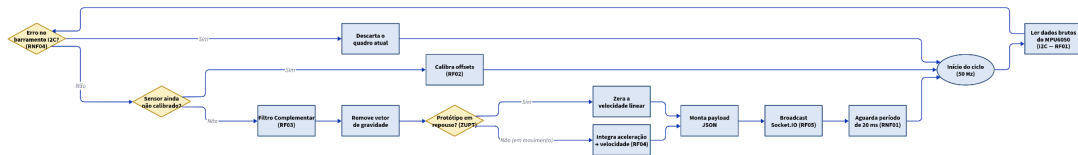


Figura 4 — Fluxograma do loop principal de aquisição e transmissão.

4.4 Backend Embarcado (backend/)

Loop principal orquestrado por main.py, executado a 50 Hz (RNF01):

- **sensor_reader.py — SensorReader:** leitura crua do MPU6050 via I2C (smbus2), com calibração de offsets em repouso (calibrate(), RF01/RF02) e tratamento de erros de barramento sem interromper o loop (RNF04).
- **orientation_filter.py — ComplementaryFilter:** fusão do acelerômetro e giroscópio ($\alpha = 0,96$) para estimar Roll, Pitch e Yaw em graus (RF03).
- **velocity_estimator.py — VelocityEstimator:** remove o vetor de gravidade da leitura de aceleração usando a orientação estimada e aplica ZUPT (Zero Velocity Update) para atenuar o drift de integração (RF04).
- **websocket_server.py — WebSocketServer:** servidor Socket.IO sobre aiohttp, mantém os clientes conectados e faz broadcast do payload no evento telemetry, porta 8765 (RF05).
- **iniciar_wifi.sh:** sobe a Raspberry Pi como Hotspot Wi-Fi autônomo (Robo-Network, gateway fixo 10.42.0.1) via nmcli, dispensando roteador externo (RNF06).

Payload transmitido a cada ciclo:

```
{
  "timestamp": 1690000000.123,
  "gyro": { "x": 0.1, "y": -0.2, "z": 1.3 },
  "accel": { "x": 0.05, "y": 0.01, "z": 9.81 },
  "linear_velocity": { "x": 0.5, "y": 0.0, "z": 0.0 },
  "orientation": { "roll": 1.2, "pitch": -0.5, "yaw": 45.0 }
}
```

4.5 Frontend / Gêmeo Digital (front/)

Aplicação Web construída com TanStack Start, Three.js e socket.io-client:

- **services/websocket.ts e hooks/useSensorSocket.ts:** conectam ao servidor e expõem os pacotes de telemetria recebidos, com suporte a modo mock e conexão real (RF08).
- **hooks/usePositionTracker.ts:** integra a velocidade linear recebida para estimar a posição do protótipo no espaço.
- **components/TwinScene, RobotTwin e TrailPath:** Cena 3D do Gêmeo Digital: aplicam a orientação ao modelo com interpolação linear no loop de animação, evitando re-renders a cada pacote recebido (RNF05, RF07).
- **routes/dashboard.tsx:** Painel com gráficos dos dados de telemetria.
- **routes/twin.tsx:** Gêmeo Digital 3D.

4.6 Comunicação

- **Transporte:** Wi-Fi local via Hotspot dedicado, usando Socket.IO na porta 8765 (RF05/RF06/RNF02).
- **Frequência:** Amostragem e transmissão a 50 Hz (RNF01); o frontend consome esse fluxo e interpola visualmente para manter fluidez (RNF05).

4.7 Justificativa das Decisões Arquiteturais

Filtro Complementar em vez de EKF: um Filtro de Kalman Estendido seria mais preciso, mas custoso demais para o ciclo de 20 ms da Raspberry Pi (RNF01). Mahony, Hamel e Pflimlin (2008) mostram estabilidade equivalente com custo bem menor, o que favorece o processamento de borda do protótipo.

ZUPT para corrigir a velocidade: a integração direta da aceleração acumula erro (drift) por causa do bias dos acelerômetros. Zerar a velocidade em períodos de repouso reduz esse drift sem sensores adicionais, atendendo ao RF04.

Processamento na borda (edge computing): enviar dados brutos à nuvem adicionaria latência e dependência de conectividade, inviável em rodovias isoladas, por isso calibração e filtragem ocorrem na própria Raspberry Pi (RNF02, RNF03).

WebSocket/Socket.IO em vez de polling HTTP: a telemetria a 50 Hz exige um canal contínuo. A RFC 6455 define o WebSocket como full-duplex sobre uma única conexão TCP, sem o overhead de requisições HTTP repetidas, viabilizando RF05 e a latência exigida em RNF02.

Separação backend/frontend como Gêmeo Digital: Uma réplica virtual sincronizada com o objeto físico, o que motivou transmitir o estado completo a cada ciclo em vez de eventos discretos (RF07).

Tratamento defensivo de erros I2C: barramentos I2C são suscetíveis a ruído em protótipos móveis. Por isso o SensorReader descarta quadros corrompidos sem interromper o loop (RNF04).

4.8 Próximos Refinamentos

- Adicionar testes automatizados de integração cobrindo o pipeline completo sensor → filtro → WebSocket → frontend.

4.9 Arquitetura física

A arquitetura física do protótipo consiste em uma Raspberry Pi, conectada ao módulo MPU6050 por barramento I2C através da protoboard FNk0054 (SDA no GPIO2/pino 3, SCL no GPIO3/pino 5, VCC no pino de 3.3V e GND no pino 6). O conjunto é utilizado para captura contínua dos dados cinemáticos.

Arquitetura Física do Protótipo

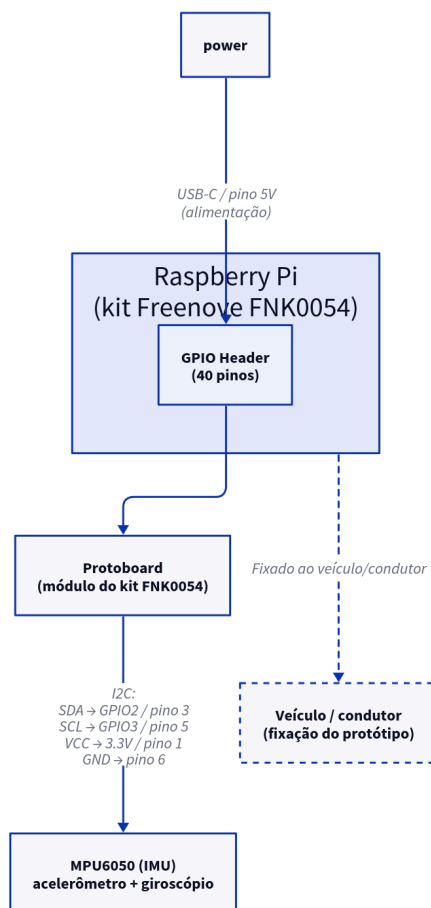


Figura 5 — Diagrama da arquitetura física da solução proposta.

5. Ferramentas Utilizadas

5.1 Linguagens

- **Python:** Linguagem principal do backend embarcado na Raspberry Pi devido ao vasto ecossistema de bibliotecas de hardware e facilidade de manipulação matemática rápida.
- **TypeScript / JavaScript (TSX):** Utilizada no desenvolvimento do frontend web para prover segurança estática de tipos nos modelos de dados de sensores.
- **Bash (Shell Script):** Usado para automatizar scripts de infraestrutura de rede no Linux embarcado.

5.2 Bibliotecas / Frameworks

Backend:

- **smbus2:** Manipulação direta das chamadas de sistema I2C do Linux, possibilitando leituras em bloco de baixo nível.
- **python-socketio & aiohttp:** Servidor HTTP assíncrono e protocolo WebSocket com suporte a salas/eventos, provendo comunicação bidirecional leve e com reconexão automática nativa.
- **asyncio:** Gerenciamento assíncrono de tarefas concorrentes (leitura do loop vs. transmissões de rede).

Frontend:

- **React 19 & TanStack Start / Router:** Framework de SPA moderno e gerenciamento de rotas/estado com baixo acoplamento.
- **Three.js (via @react-three/fiber e @react-three/drei):** Engine 3D para web que permite criar cenas complexas no WebGL com componentes declarativos React.
- **Tailwind CSS:** Estilização rápida e responsiva do dashboard.
- **Canvas 2D (customizado):** Utilização de um componente de gráficos de linha personalizado (CanvasLineChart) que desenha dados diretamente em um elemento HTML Canvas. Isso evita o uso de bibliotecas de gráficos React tradicionais (como Recharts), que forçam re-renders constantes e causariam travamento na UI devido à alta frequência de atualização (10Hz).

5.3 Hardware

- **Unidade de Processamento:** Raspberry Pi equipada com Wi-Fi onboard rodando sistema operacional Linux (Debian/Raspberry Pi OS).
- **Sensor Inercial:** MPU6050 (Unidade de Medição Inercial de 6 graus de liberdade contendo acelerômetro triaxial microeletromecânico e giroscópio triaxial).
- **Conectividade Física:** Placa de prototipagem (Protoboard) e jumpers macho-fêmea para barramento I2C (pinos SDA/SCL, alimentação 3.3V/GND).
- **Fonte de Energia:** Bateria externa (Power Bank 5V/3A) para alimentação via barramento USB-C ou pinos de energia GPIO.

6. Metodologia de Desenvolvimento

Adotou-se o modelo de desenvolvimento baseado em Prototipagem Rápida e Arquitetura Descentralizada Edge-Cloud, com os seguintes marcos:

- **1. Isolamento do Frontend (Mocking First):** Desenvolvimento inicial de toda a interface visual 3D e gráficos em TypeScript utilizando geradores de telemetria artificial. Isso permitiu otimizar o fluxo de renderização e animações 3D de forma independente do hardware físico.
- **2. Desenvolvimento do Driver do Hardware:** Escrita do módulo de captura I2C em Python, testando e validando as conversões de escalas e o tratamento de ruídos físicos em repouso.
- **3. Refinamento Matemático:** Implementação de protótipos rápidos dos filtros de orientação e velocidade para análise do comportamento dos eixos e mitigação do desvio (drift).
- **4. Integração e Testes Remotos:** Configuração do ponto de acesso Wi-Fi nativo e união das duas frentes pelo protocolo Socket.IO.

7. Testes Planejados e Resultados Obtidos

Nas atividades do projeto, o foco foi direcionado à criação de novos testes de unidade e à automatização deles via CI no GitHub, à criação de uma forma de exportar os dados para formatos mais legíveis (.csv) e à resolução de um bug no qual o frontend sofria pequenas travadas gráficas pontuais ao receber os dados vindos da Raspberry Pi. Além disso, foi implementada uma camada de observabilidade no backend, com logs, métricas de CPU/RAM, latência do loop de controle e RTT de rede, expostas por um novo painel de saúde do sistema (/health), e uma política de reconexão automática (backoff exponencial) para lidar com quedas momentâneas de sinal Wi-Fi. A correção de estabilidade do frontend passou pelo reagrupamento da telemetria transmitida (de 50 Hz para lotes a 10 Hz), pela substituição dos gráficos SVG por renderização em Canvas

e pelo isolamento do ciclo de vida do WebSocket por rota, eliminando os travamentos e a perda de contexto observados em sessões mais longas.

7.1 Testes de Execução Isolada (Backend)

- **Script de Teste (main_test.py):** Objetivo: Validar a leitura e o processamento cinemático localmente na Raspberry Pi, sem a necessidade de um cliente web conectado. Resultado: O script calibra o sensor com sucesso e imprime no terminal a aceleração corrigida, giroscópio, ângulos calculados e velocidade linear.
- **Script de Cliente de Teste (client_test.py):** Objetivo: Validar a conectividade WebSocket e integridade estrutural do JSON transmitido. Resultado: Conecta-se no IP fixo, recebe o evento telemetry e imprime os dados. Validou a estabilidade e ausência de estouros de memória.

7.2 Tratamento de Anomalias Físicas

- **Ruído de Conexão I2C:** Testes desconectando os fios SDA/SCL temporariamente demonstraram que o tratamento de exceções descarta o frame corrupto sem interromper a execução do servidor e retoma a leitura assim que o contato físico é reestabelecido.
- **Drift de Integração e ZUPT:** O movimento livre do sensor gera rápido acúmulo de erro na velocidade integrada. A calibração estática removeu o bias e o algoritmo ZUPT mostrou-se eficaz ao forçar a velocidade de volta a zero após o protótipo permanecer parado por 5 quadros consecutivos.

7.3 Performance de Visualização

A frequência de 50 Hz sobrecarregaria o React se o estado global fosse atualizado a cada pacote. A solução de acumulação em lotes de 5 frames (10Hz de transmissão) aliada à interpolação por quadro de animação (lerp/slerp) no Three.js estabilizou o consumo de CPU do navegador abaixo de 10% em computadores modernos, com movimentação do modelo 3D fluida a 60 FPS.

7.4 Interface Visual do Sistema

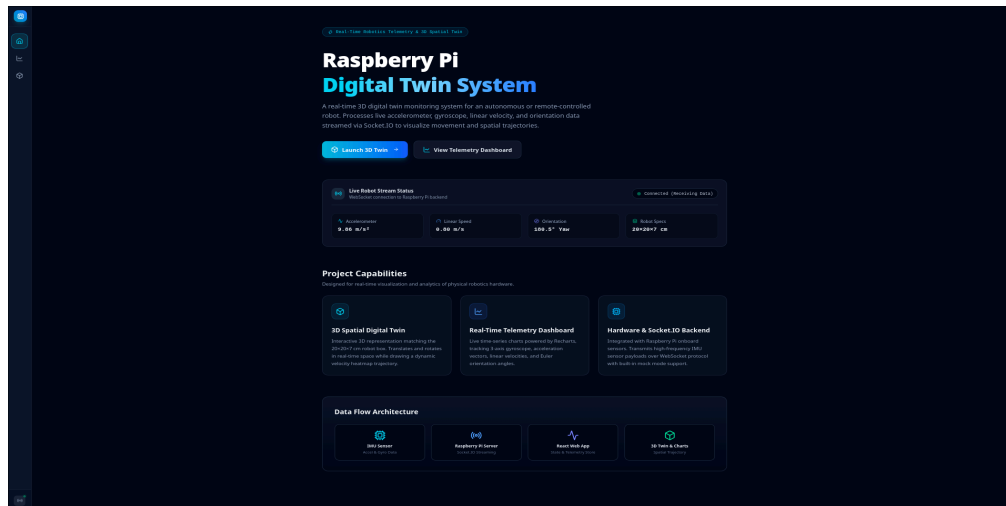


Figura 5: Página Inicial do sistema com status de conexão



Figura 6: Dashboard de telemetria em tempo real

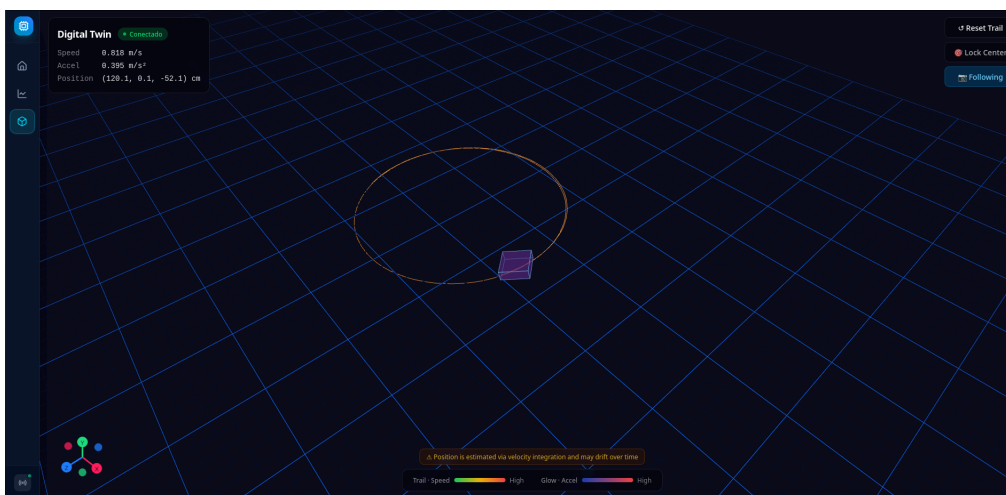


Figura 7: Visualização 3D do Gêmeo Digital com o rastro estimado do protótipo

8. Atividades de Peer Review

Resumo das issues abertas para a atividade de peer review: duas issues criadas em repositórios de colegas, e o panorama das issues abertas em nosso próprio repositório (rpi-digital-twin).

8.1. Issue criada — WesleyPoli/LabProc-ProjetoFinal #2

Título: Alerta sonoro ao detectar obstáculo próximo

Link: <https://github.com/WesleyPoli/LabProc-ProjetoFinal/issues/2>

Estado: Aberta

Propõe integrar um buzzer ao carrinho: quando o sensor ultrassônico (HC-SR04) detectar um obstáculo dentro da distância crítica de segurança, o buzzer emite alerta sonoro intermitente, desligando automaticamente quando o obstáculo se afasta.

Critérios de aceite: buzzer aciona dentro da distância crítica; desliga ao afastar o obstáculo; não atrasa a parada automática dos motores; não compromete a latência de controle remoto.

8.2. Issue criada — LABPROC2026GA/P2-GA-WorkTrack #2

Título: Falta de opção de remoção de usuário

Link: <https://github.com/LABPROC2026GA/P2-GA-WorkTrack/issues/2>

Estado: Aberta

Aponta que o sistema não possui funcionalidade para deletar usuários pela aplicação. Propõe um endpoint de exclusão por ID único com validação de permissões e tratamento de erros.

Critérios de aceite: exclusão por ID; remoção não afeta outros usuários; erros tratados adequadamente; testes cobrindo sucesso, usuário inexistente e acesso não autorizado.

8.3. Feedback recebido — gabrielsoares01/rpi-digital-twin #16

Título: Falta de automação no GitHub (CI): risco de subir código com bug ou desconfigurado

Link: <https://github.com/gabrielsoares01/rpi-digital-twin/issues/16>

Estado: Fechada

Issue aberta por membros de outro grupo como feedback de peer review, apontando a ausência de um pipeline de CI/CD no repositório.

8.4. Feedback recebido — gabrielsoares01/rpi-digital-twin #18

Título: Exportar histórico de telemetria em CSV

Link: <https://github.com/gabrielsoares01/rpi-digital-twin/issues/18>

Estado: Aberta

O dashboard já mantém em memória um histórico recente das leituras de telemetria. A issue propõe permitir a exportação desse histórico em CSV para análise posterior.

8.5. Issues abertas no nosso repositório — gabrielsoares01/rpi-digital-twin

Link: <https://github.com/gabrielsoares01/rpi-digital-twin/issues>

#	Título	Labels	Resumo
15	Adicionar observabilidade (logs/métricas) e recuperação de falhas	enhancement	Propõe logging estruturado, exposição de métricas, políticas de retry/backoff e um dashboard de saúde do sistema
14	Validar consistência entre requisitos e diagramas de arquitetura	documentation	Pede uma matriz de rastreabilidade ligando requisitos aos componentes de arquitetura e aos testes
13	Requisitos estão incompletos/ambíguos: detalhar critérios de aceite e NFRs	documentation	Pede critérios de aceite explícitos, requisitos não funcionais mensuráveis e documentação de restrições de hardware
12	Criação de Testes e Testes Automatizados	enhancement	Pede testes unitários, de integração, de carga e de regressão, automatizados via GitHub Actions
11	Frontend trava ao receber stream de dados	bug, enhancement	UI trava durante recebimento contínuo de telemetria; sugere debouncing, buffering e profiling para otimização

9. Conclusões

9.1 Objetivos Cumpridos e Requisitos Satisfeitos

Todos os objetivos gerais e específicos definidos foram alcançados. Os requisitos funcionais RF01 a RF08 foram completamente satisfeitos e testados, assim como os requisitos não funcionais de performance de rede (latência < 100ms no hotspot local), frequência de leitura estável a 50 Hz e renderização 3D livre de gargalos de CPU (RNF01 a RNF06).

9.2 Dificuldades Encontradas e Lições Aprendidas

- **Desvio da Velocidade Linear (Drift):** O cálculo de velocidade por integração numérica de dados acelerométricos sem sensores adicionais de referência acumula erros exponencialmente. O uso de ZUPT foi crítico para conter o drift, fixando o zero absoluto em paradas temporárias.
- **Comunicação I2C no Linux:** A Raspberry Pi compartilha recursos que podem induzir leves latências no barramento I2C, gerando erros ocasionais. A modularização do SensorReader isolando as leituras cruas protegeu o loop principal.
- **Gerenciamento de Estado no React:** O fluxo massivo de dados de sensores colapsa o virtual DOM. O uso de componentes puros (memo), hooks sincronizados externamente (useSyncExternalStore), buffers de array tipados estáticos reutilizados na memória e a substituição do Recharts por renderização nativa em Canvas 2D provaram ser as melhores táticas para dashboards de alta performance.

9.3 Trabalhos Futuros

- **1. Algoritmo Dedicado de Detecção de Acidentes:** Implementar no backend (em Python) a lógica de identificação de acidentes baseada nas assinaturas de impacto (desaceleração rápida em X/Y), queda de velocidade linear e orientação anômala persistente (Roll ou Pitch fora da faixa permitida).
- **2. Fusão Sensorial Expandida:** Integrar um magnetômetro (ex: MPU9250) para aplicar algoritmos de orientação mais complexos (como filtro de Kalman, Madgwick ou Mahony) a fim de conter o drift do eixo Yaw (azimute).
- **3. Modelagem Realística 3D:** Substituir o bloco representativo por um modelo CAD tridimensional realístico de um veículo de duas rodas (motocicleta) ou robô móvel para melhor apelo visual do Gêmeo Digital.
- **4. Integração de Módulo GPS e Conectividade Celular para Rastreamento e Resgate Remote:** Incorporar um receptor GPS acoplado a um módulo de comunicação via rede móvel (SIM card), viabilizando o envio à distância das coordenadas geográficas exatas e acelerando a localização de eventuais ocorrências de acidentes em áreas isoladas.

Apêndice

[Link do repositório público](#)

[Imagem com uma amostra de leitura do sensor](#)

[Evidencias dos testes feitos no laboratório](#)

Referências

EUROPEAN COMMISSION. eCall in all new cars from April 2018. Shaping Europe's Digital Future, Bruxelas, 28 abr. 2015. Disponível em: <https://digital-strategy.ec.europa.eu/en/news/ecall-all-new-cars-april-2018>. Acesso em: 20 ago. 2026.

APPLE INC. Use Crash Detection on iPhone or Apple Watch to call for help in an accident. Apple Support, Cupertino, 2022. Disponível em: <https://support.apple.com/en-us/104959>. Acesso em: 20 ago. 2026.

MAHONY, R.; HAMEL, T.; PFLIMLIN, J.-M. Nonlinear complementary filters on the special orthogonal group. IEEE Transactions on Automatic Control, v. 53, n. 5, p. 1203-1218, jun. 2008. DOI: 10.1109/TAC.2008.923738.

FETTE, I.; MELNIKOV, A. The WebSocket Protocol. RFC 6455. Fremont: Internet Engineering Task Force (IETF), dez. 2011. Disponível em: <https://www.rfc-editor.org/rfc/rfc6455>. Acesso em: 20 ago. 2026.